

Multi-Tenant School ERP

One platform running an entire group of schools — each isolated, each on its own plan.

Sunil Kushwaha · Full Stack Developer (MERN) · Oct 2024 – Sep 2026

210K+

LINES OF CODE

1,284

COMMITTS

165

DATA MODELS

108

API MODULES

327

SCREENS

THE PROBLEM

Private schools run admissions on paper, fees in a cash book, attendance in registers and payroll in spreadsheets. Nothing reconciles. A principal asking how much fee is outstanding this month, or which students are below 75% attendance, cannot answer it without a week of manual collation.

Off-the-shelf ERPs are priced per school and sold as single-tenant installs, so a group running several branches ends up with several disconnected systems and no consolidated view.

WHAT I BUILT

A multi-tenant SaaS platform where one deployment serves unlimited schools. Every record is scoped to its school; each gets its own administrator, roles, academic year and subscription, and none can see another's data. I built it end to end — database design, API, web app, React Native companion app, payments and deployment — in continuous development since October 2024.

- **Tenant isolation** enforced at the query layer, not the UI — every controller resolves the caller's school from their token, and a hand-crafted request for another school's data is refused rather than filtered.
- **Role-based access** across 24 roles from Super Admin to Parent. Each has its own dashboard and navigation tree, not one admin panel with fields hidden.
- **Subscription billing** with plan-based module gating — a school that has not bought the Transport module sees it locked, with an upgrade path, rather than missing.

Modules

Fifteen functional areas, each a working workflow rather than a CRUD screen.

Admissions & Student Lifecycle

Inquiry tracking, multi-step admission with live roll-number preview, document upload, bulk promotion between academic years.

Fees & Finance

Fee heads, per-class structures, instalment schedules, concessions, late fines, Razorpay collection with webhook reconciliation, PDF receipts.

Examinations

Exam scheduling, seat plans, admit cards, marks entry, grading scales, report cards from configurable templates, analytics.

Library

Catalogue, issue and return, fines, member activity, per-school lending rules.

HR

Employee records, departments, designations, leave workflow, recruitment pipeline, appraisal cycles.

Inventory & Procurement

Items, vendors, purchase orders, stock issue, asset tracking, AMC schedules.

Attendance

GPS geofencing for staff check-in, biometric/RFID device integration, auto-checkout job, configurable late-grace window.

Payroll

Salary structures, monthly runs, payslip generation, advances, bonuses, reimbursements, PF/ESI reports.

Transport

Routes, vehicles, driver assignment, live GPS bus tracking on a Leaflet map, maintenance and fuel logs.

Hostel

Blocks, rooms, bed allocation, hostel attendance, leave and visitor logs, complaints.

Communication

In-app notifications, SMS via Twilio, email via SMTP, push via Firebase, circulars with read receipts.

Compliance

UDISE+ identifiers, RTE quota tracking, readiness reporting showing exactly which records are incomplete.

ENGINEERING DECISIONS WORTH CALLING OUT

- **Money is never recomputed on read.** An issued fee receipt stores the figures it was issued with. Changing a fee structure later reprices the next student, not one already invoiced.
- **Instalments always sum to the total.** Splitting a fee three ways rounds every part down and lets the last carry the remainder — otherwise the bill is short by a rupee and never clears.
- **Absent is not zero.** An unmarked paper and a paper scored zero are stored differently, so a student who has not sat an exam is never reported as having failed it.
- **Role assignment is server-enforced.** A School Admin cannot mint a Super Admin by posting a role id they can read — the API refuses roles outside their own school.
- **Security throughout.** JWT with refresh rotation, 2FA/OTP, bcrypt, rate limiting, helmet, Mongo injection and XSS sanitisation, IP restriction, and an audit log on every mutation.

Technical architecture

FRONTEND	React 19, Redux Toolkit, Ant Design, React Router 7, Recharts, Leaflet, Vite. 327 screens across 24 role-driven navigation trees.
BACKEND	Node.js, Express, MongoDB with Mongoose. 165 models, 113 controllers, 108 route modules, aggregation pipelines for every report.
MOBILE	React Native / Expo companion app sharing the same API.
PAYMENTS	Razorpay — per-school merchant credentials, checkout, and HMAC-verified webhooks reconciled against raw request bytes.
INTEGRATIONS	Twilio (SMS), Firebase Admin (push), Nodemailer (email), Cloudinary (media), PDFKit (receipts, ID cards, report cards), ExcelJS (exports).
JOBS	node-cron for subscription expiry, fee-overdue notices, auto-checkout and PTM reminders.

SCALE OF THE WORK

23 MONTHS	1,284 COMMITTS	1,153 SOURCE FILES	24 ROLES
--------------	-------------------	-----------------------	-------------

The commit history is incremental and continuous — this was built and iterated in production conditions, not generated in a weekend.

WHAT THIS DEMONSTRATES

If you are hiring for a product that needs more than screens on a database: multi-tenancy at the data layer, money that reconciles, permissions that hold when someone pokes at the API, and a codebase that stayed maintainable across two years and a thousand commits.

Available for similar work

Multi-tenant SaaS · payment and billing systems · role-based platforms · MERN applications at scale